

Vellore Institute of Technology

School of Computer Science and Information Systems

Fall Semester 2026–27

Course Name: Software Metrics

PROJECT – FINAL REPORT

Digital Fugitive

Simulating AI-Driven Surveillance and Logical Inference in a Grid Environment

TEAM MEMBERS

S. No.	Register Number	Name
1	23MIS0296	Sooriya Moorthy
2	23MIS0527	Deeksha M
3	23MIS0280	Thiru Dev B
4	23MIS0521	Priyadharshini S

1. Problem Statement

Locating hidden fugitives in a large search area is a search-and-decision problem: the agent does not initially know which cells contain targets and must use observations to decide where to move next. A purely random strategy can repeatedly inspect unhelpful locations, increase the number of moves, and make the search process difficult to explain.

The Digital Fugitive project models this challenge as a 16 × 16 grid. Each cell represents a possible location, and neighbouring-cell information, the current search state and Hot/Cold radar feedback help guide the agent. The system compares randomized exploration with knowledge-based reasoning that uses a knowledge base, propositional logic and constraint propagation.

The project also needs an objective way to evaluate the behaviour of the simulation. Therefore, 128 recorded runs are examined using measures such as cells inspected, safe moves, fallback/random moves, targets identified, total moves, run outcome and execution time. The central problem is to demonstrate how the search process works and use software metrics to describe its effort, variation and outcomes.

2. Introduction

Digital Fugitive is an interactive Pygame simulation designed to demonstrate AI-assisted searching in a grid environment. Hidden fugitives are placed in a 16 × 16 grid, and the agent explores the grid while the interface displays cell states and radar feedback. The interface makes the process visible: users can see the grid, target placements in the setup phase, and the changing cell colours and symbols during radar tracking.

The search process combines two broad approaches. Randomized exploration provides a fallback when a guaranteed safe choice is not available. Knowledge-based search uses the observations gathered so far, a record of visited/safe/suspected cells, neighbouring-cell relationships and Manhattan-distance-based Hot/Cold feedback to prioritize the next action. Propositional logic and constraint propagation help eliminate candidate cells that conflict with the current evidence.

Alongside the simulation, the project applies software-metrics techniques to a dataset of 128 runs. Box plots describe distributions, a control chart summarizes variation in total moves, Pearson/Spearman/Kendall matrices examine associations among measures, and a decision tree classifies the recorded outcome as Failure or No Failure. These analyses provide a structured view of the behaviour recorded by the supplied project dataset.

3. Abstract

This project presents Digital Fugitive, an interactive AI search simulation that models the task of locating hidden fugitives in a 16 × 16 grid. The Pygame interface allows a scenario to be configured and displays the search state through cell markings, radar feedback and heat-map-style colours. The search process uses randomized exploration and knowledge-based reasoning. The knowledge base records visited, safe and suspected cells; propositional logic and constraint propagation use observed evidence to eliminate inconsistent candidates; and Manhattan distance supports Hot/Cold proximity feedback. When the system cannot identify a guaranteed safe move, fallback exploration allows the search to continue.

The software-metrics component evaluates 128 recorded runs using box plots, a control chart, Pearson, Spearman and Kendall correlation analyses, and decision-tree classification. The supplied report records a mean of 43.08 total moves, with control limits of 83.49 (UCL) and 2.66 (LCL). It also reports a correlation of 1.00 between cells inspected and total moves, and strong positive associations between search effort and execution time. The decision tree uses run-level metrics to classify Failure versus No Failure. Together, the simulation and analysis make search behaviour visible and measurable. Results are specific to the supplied simulation and dataset; correlations describe association rather than proving causation.

4. Summary and Motivation

Summary. Digital Fugitive combines an interactive grid-based search simulation with a software-metrics evaluation. The simulation represents the search environment, records evidence as cells are inspected, uses logical reasoning and radar feedback to guide actions, and presents the resulting behaviour visually. The analysis component studies 128 runs using distribution plots, a control chart, three correlation methods and a decision-tree classifier. This combination links the system's internal decision process to measurable outcomes such as inspected cells, total moves and execution time.

Motivation. The project is motivated by the need to make an AI search process understandable instead of treating it as a black box. A visible grid allows users to follow how evidence changes the set of candidate cells, while radar and cell-state visuals provide an intuitive explanation of proximity and search progress. Repeated runs make it possible to compare outcomes and investigate whether more extensive searches tend to involve more moves or longer execution times. The metrics therefore support evidence-based discussion of the simulation's behaviour, its limitations and possible improvements.

5. Proposed System

The proposed system uses an interactive 16 × 16 grid. Users place hidden fugitives and start the simulation. The AI tracks explored cells, updates its knowledge, uses local and radar evidence to classify candidate cells, and chooses the next move. When a guaranteed safe move cannot be identified, fallback exploration allows the search to continue. The interface presents grid state and feedback, while a performance evaluator records run-level metrics.

- Interactive Pygame interface for configuring and running scenarios.
- Grid environment with cell and neighbouring-cell relationships.
- Randomized exploration and knowledge-based search modes.
- Knowledge base for explored, safe and suspected cells.
- Propositional logic and constraint propagation to eliminate inconsistent candidates.
- Manhattan-distance Hot/Cold radar for proximity feedback.
- Run-level logging and statistical analysis across 128 executions.

6. Methodology and Classification Used

6.1 Execution Workflow

The workflow below presents the execution sequence in a step-by-step format, from scenario initialization to recording metrics. The reasoning and feedback loop is repeated until the target is found or the run ends.

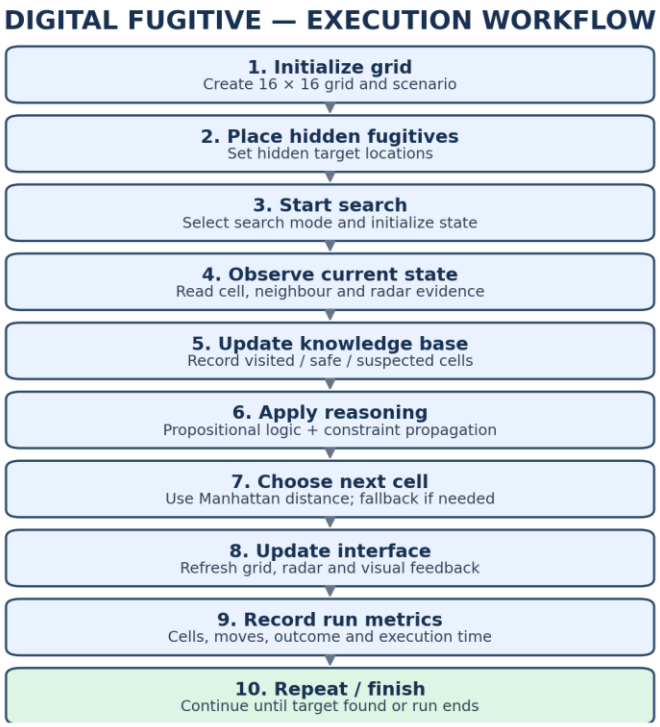


Figure 1. Digital Fugitive execution workflow

6.2 AI Techniques

Random search selects unexplored cells without using the full accumulated reasoning state. Knowledge-based search uses observations and a knowledge base to classify cells and guide the next action. Propositional logic represents constraints about possible cell states. Constraint propagation transfers implications to related cells and removes inconsistent candidates. Manhattan distance measures grid distance between positions and supports proximity-based prioritization. The Hot/Cold radar gives visual feedback about closeness to a hidden target.

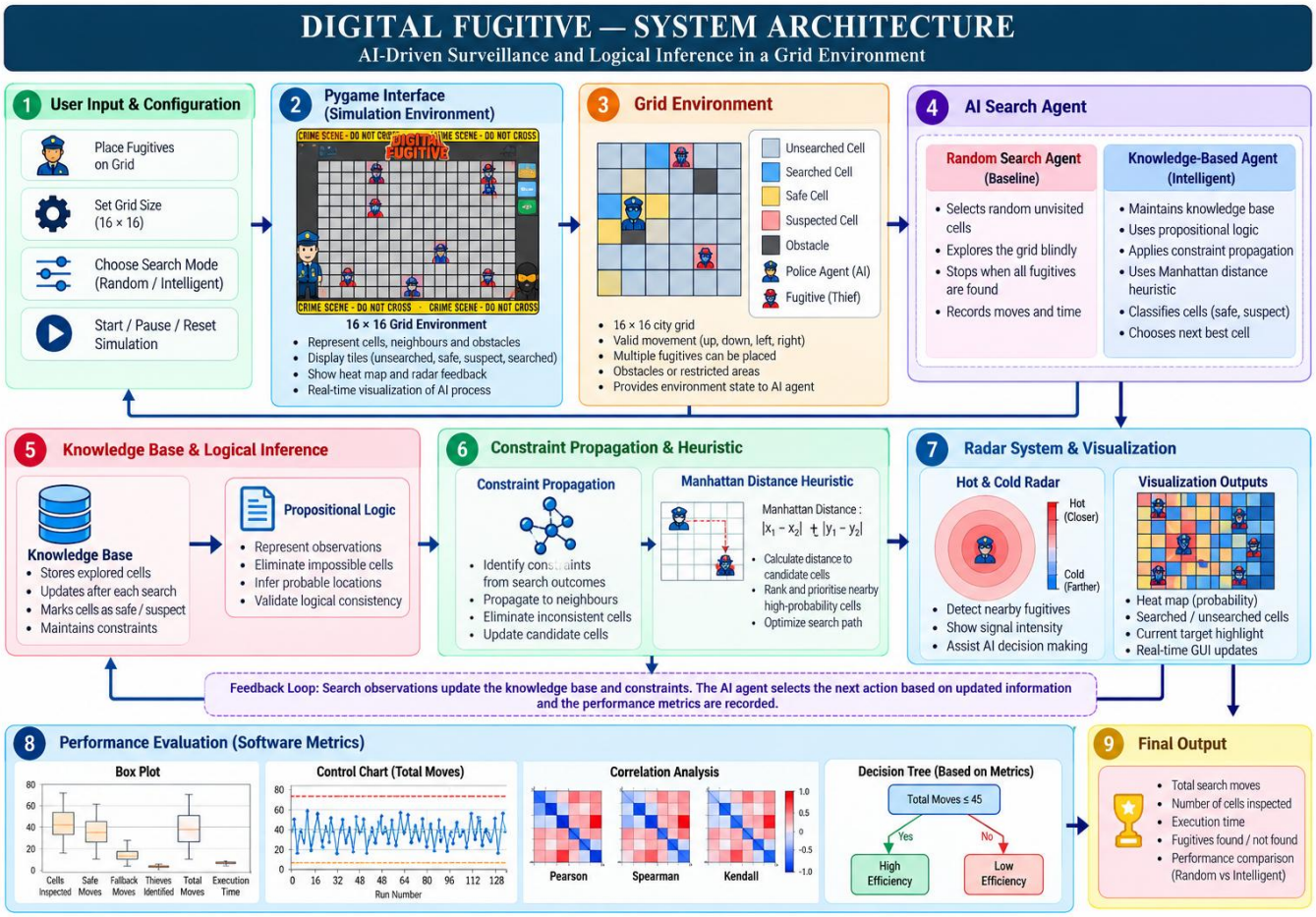
6.3 Software-Metrics and Classification Methods

Technique	Purpose	Reported use / result
Box plot	Summarize distribution, median and spread.	Cells Inspected and Total Moves show broad variation; execution time varies comparatively little.
Control chart	Observe Total Moves across 128 runs.	Mean = 43.08; UCL = 83.49; LCL = 2.66; reported points remain within limits.
Pearson correlation	Measure linear relationships.	Cells–Moves = 1.00; Cells–Time = 0.98; Moves–Time = 0.98.
Spearman correlation	Measure rank-based association.	Cells–Moves = 1.00; Cells–Time = 0.97; Moves–Time = 0.97.
Kendall correlation	Check rank association using a second statistic.	Cells–Moves = 1.00; Cells–Time = 0.87; Moves–Time = 0.87.
Decision tree	Classify Failure vs No Failure.	Uses fallback moves, total moves, cells inspected and safe moves.

The decision tree is identified in the supplied analysis as the most directly useful classification technique because the main outcome of interest is whether a run ends in Failure or No Failure. It is a classifier built from recorded run metrics, not the sole AI search algorithm.

7. System Architecture

The architecture keeps the main flow from the original design: user controls lead to the Pygame interface, then to the grid environment and AI search agent. The search agent consults and updates the knowledge base; the inference engine applies logic and constraints; radar feedback supports proximity decisions; and visualization returns the current state to the user. The performance evaluator receives run information and records cells inspected, moves, outcomes and execution time.



.

7.1 Architecture Flow Explained

1. User controls and interface: the user places fugitives and starts the simulation through the Pygame interface.
2. Grid environment: the 16 × 16 grid stores cell states and the relationships between neighbouring cells.
3. AI search agent: the agent selects randomized or knowledge-based search behaviour and chooses a candidate cell.
4. Knowledge base: visited, safe and suspected cells are recorded so the system can use the evidence already collected.
5. Inference engine: propositional logic and constraint propagation evaluate the current evidence and eliminate inconsistent candidates.
6. Hot/Cold radar: Manhattan distance provides proximity feedback to support selection of the next cell.
7. Visualization: the grid, heat-map colours and cell symbols display the current search state to the user.
8. Performance evaluator: run-level measures—including cells inspected, safe and fallback moves, targets identified, total moves, outcome and execution time—are recorded for later analysis.
9. Feedback loop: the next observation updates the knowledge base, and the search-and-measure cycle repeats until the run ends.

8. Dataset Link and Data Collection

Dataset name: Digital Fugitive 128-Run Dataset with Execution Time

Format: CSV. The supplied project review states that the dataset was collected from repeated runs on a 16 × 16 grid. Fields include grid size, hidden thieves, cells inspected, safe moves, fallback/random moves, thieves identified, total moves, failure and failure location.

Dataset link:
https://drive.google.com/file/d/1uKCd2XEqziQDHPWwrbL6zTwGfSY_vXxP/view?usp=drivesdk

GitHub repository: https://github.com/sooriya-moorthy-0107/Digital-Fugitive.git

9. Performance Metrics and Experimental Outcomes:

9.1 Box Plot

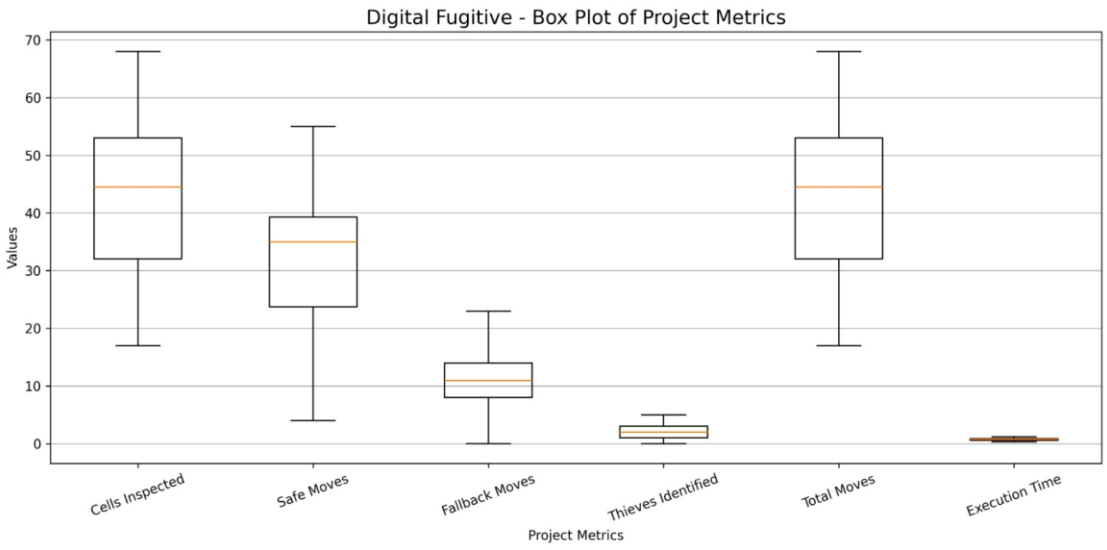


Figure 3. Box plot of Digital Fugitive project metrics.

The supplied report states that Cells Inspected ranged from 17 to 68 (median approximately 45), Safe Moves from 4 to 55 (median approximately 35), Fallback Moves from 0 to 23 (median approximately 11), Thieves Identified from 0 to 5 (median approximately 2), and Total Moves from 17 to 68 (median approximately 45). Execution time is concentrated around 0.5–1.1 seconds.

9.2 Control Chart

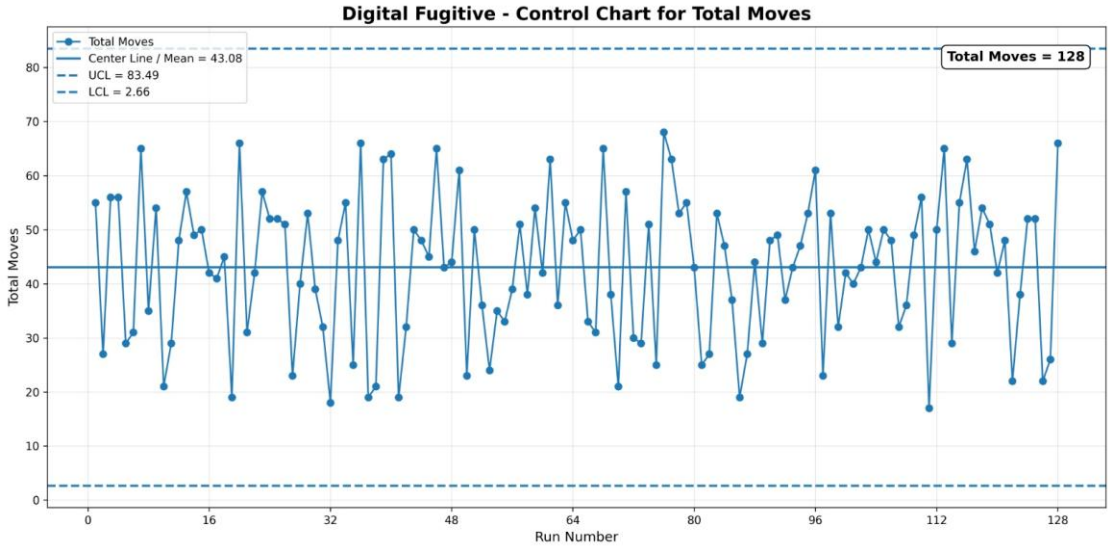


Figure 4. Control chart of Total Moves across 128 runs.

The reported mean is 43.08 moves, with UCL = 83.49 and LCL = 2.66. The report states that all plotted runs remain within the calculated control limits.

9.3 Pearson Correlation

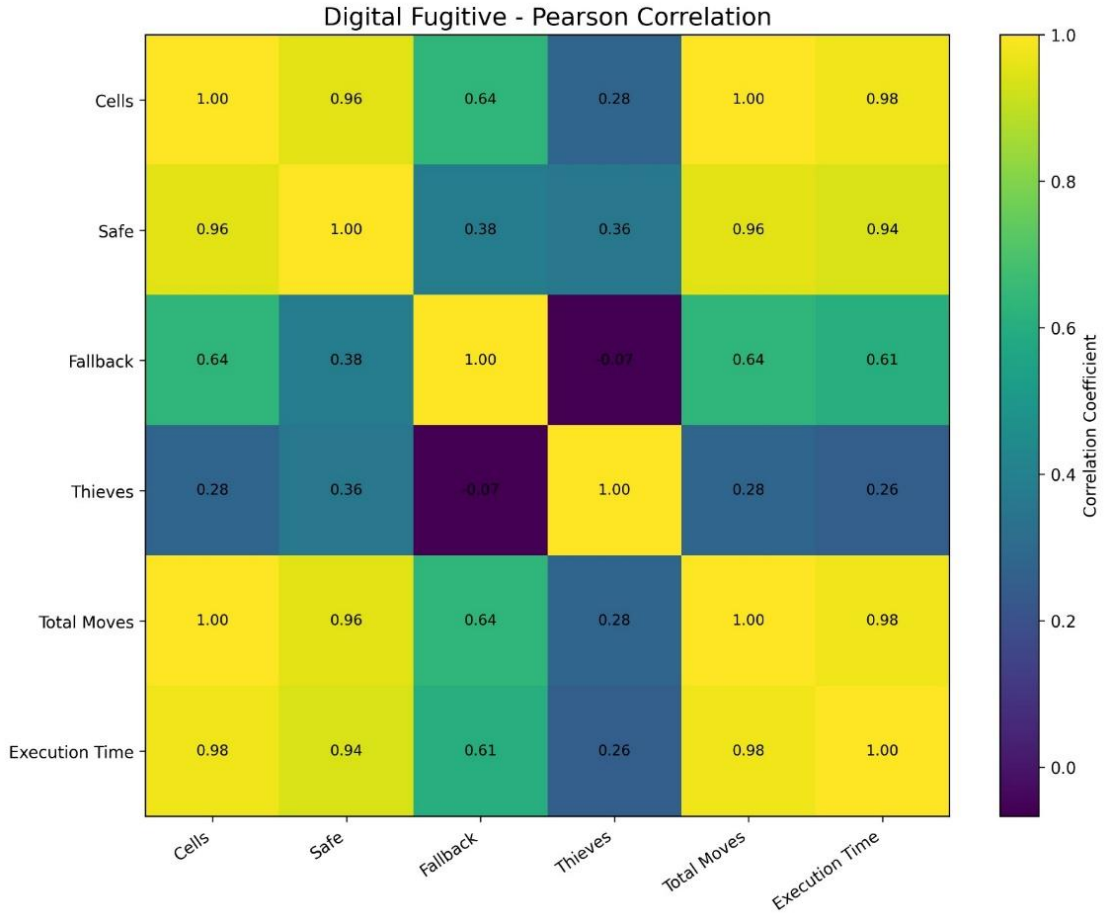


Figure 5. Pearson correlation matrix of Digital Fugitive metrics.

Cells–Total Moves = 1.00, Cells–Execution Time = 0.98 and Total Moves–Execution Time = 0.98. Fallback Moves–Thieves Identified = −0.07, indicating almost no linear association in the reported results.

9.4 Spearman Correlation

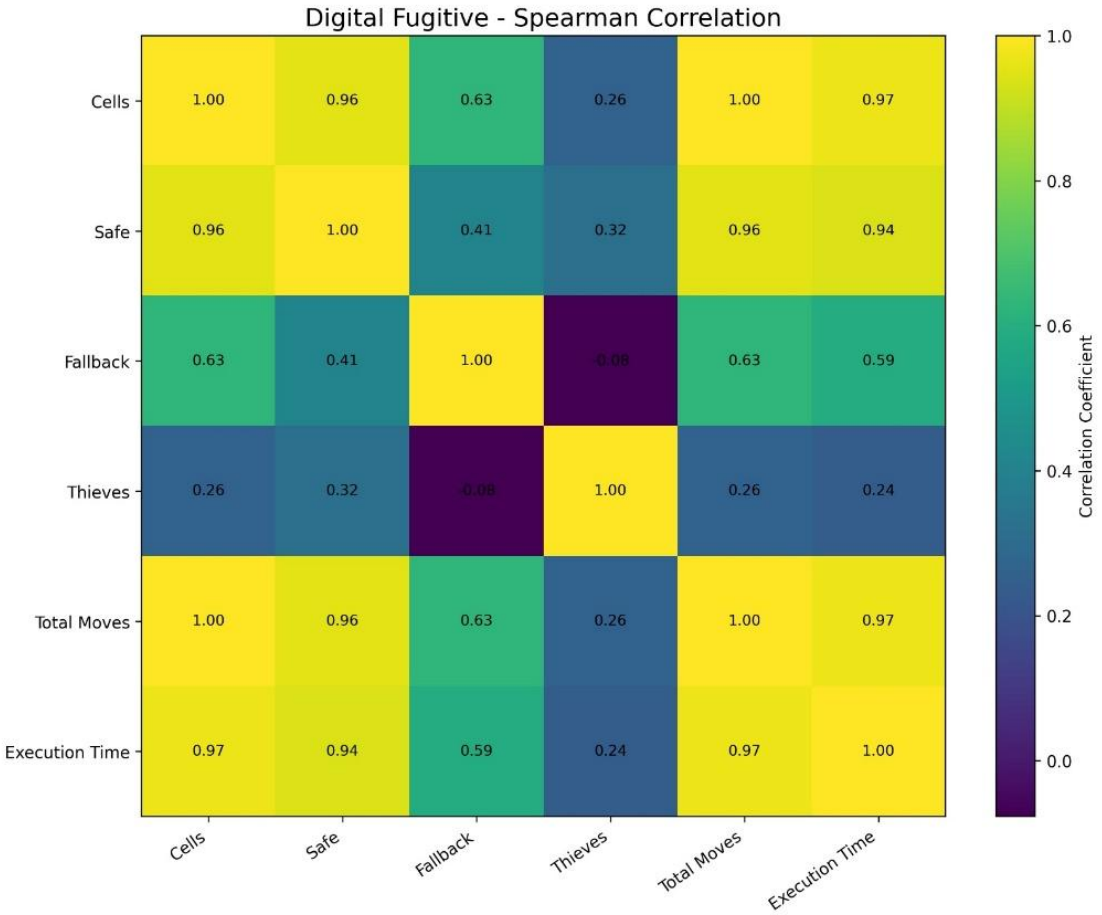


Figure 6. Spearman correlation matrix of Digital Fugitive metrics.

Cells–Total Moves = 1.00, Cells–Execution Time = 0.97 and Total Moves–Execution Time = 0.97. Fallback Moves–Thieves Identified = −0.08. This pattern is similar to the Pearson results.

9.5 Kendall Correlation

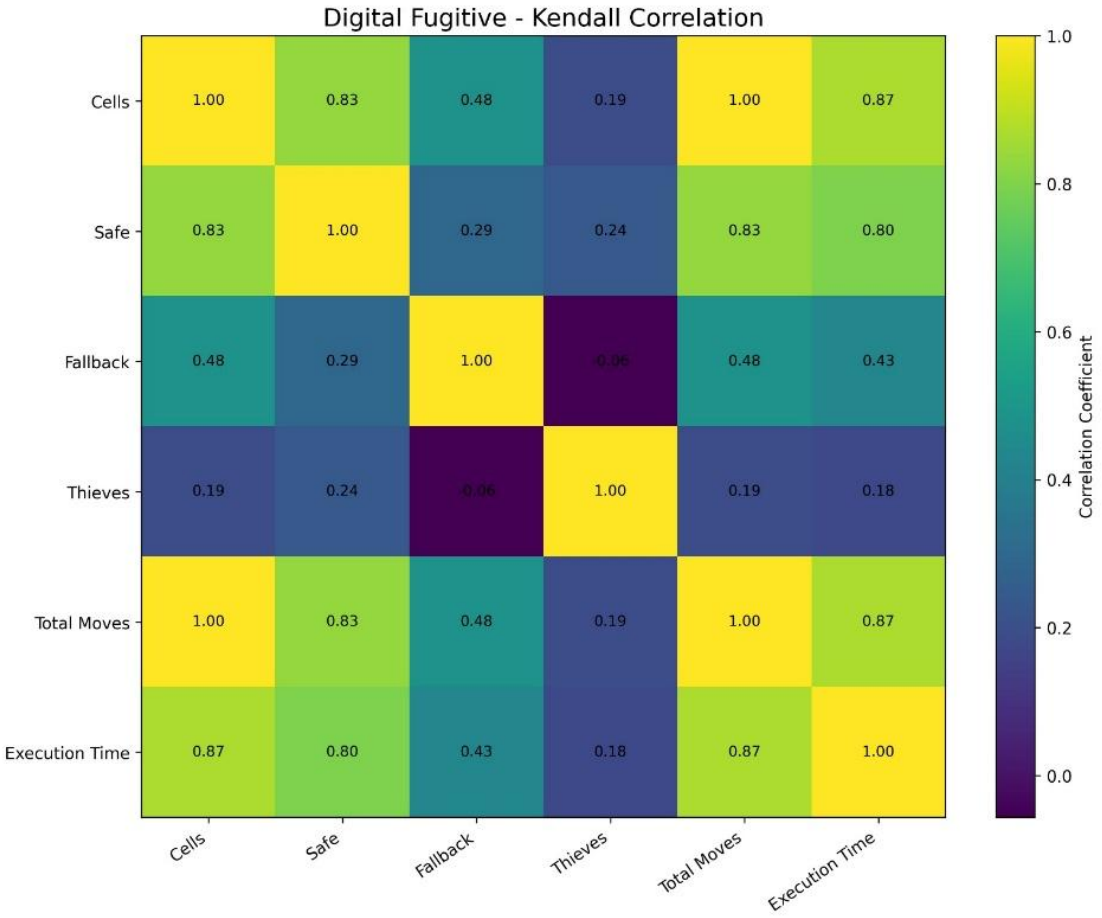


Figure 7. Kendall correlation matrix of Digital Fugitive metrics.

Cells–Total Moves = 1.00, Cells–Execution Time = 0.87 and Total Moves–Execution Time = 0.87. Fallback Moves–Thieves Identified = −0.06. This provides a rank-based comparison of the relationships.

9.6 Decision Tree Classification

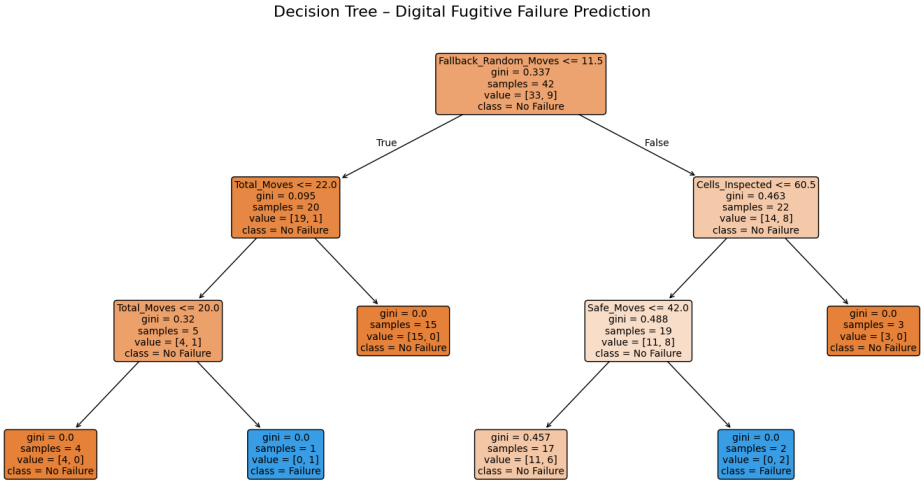


Figure 8. Decision tree for predicting Failure vs No Failure.

The supplied report identifies Fallback_Random_Moves ≤ 11.5 as the root condition. Additional splits use Total_Moves ≤ 22.0, Cells_Inspected ≤ 60.5 and Safe_Moves ≤ 42.0. The leaf nodes classify the outcome as Failure or No Failure.

9.7 Bayesian Cell-Safety Flowchart

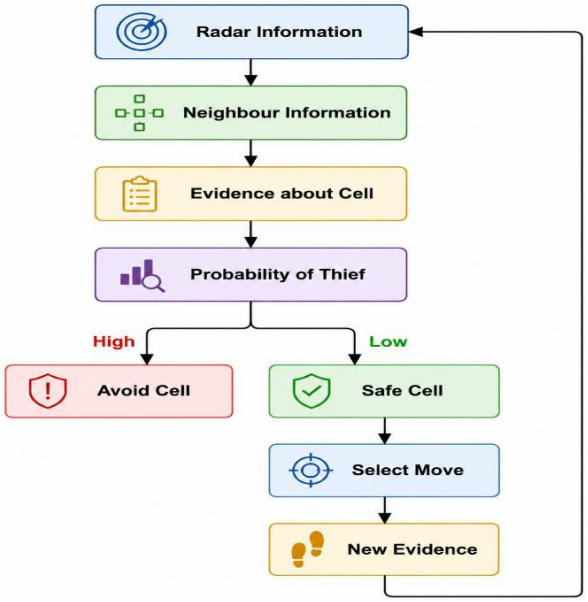


Figure 9. Bayesian cell-safety flowchart from the supplied project review.

The flowchart illustrates how radar and neighbouring-cell evidence contribute to an estimate of whether a cell contains a thief. A high estimated probability leads the AI to avoid the cell, while a lower probability allows it to remain a candidate. The next move produces new evidence for the following decision.

9.8 Grid-Based Network Analysis

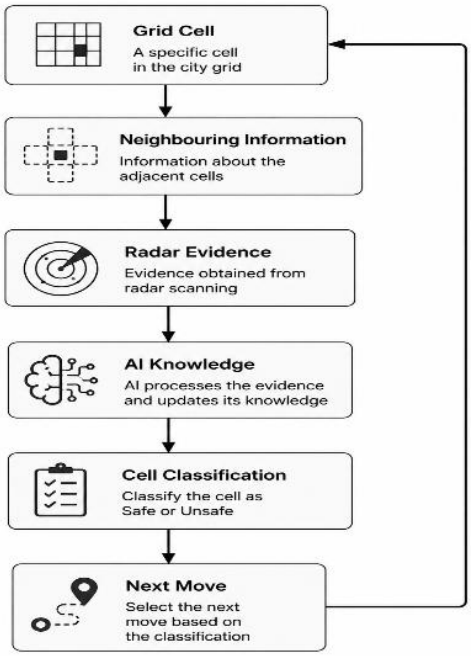


Figure 10. Grid-based network analysis from the supplied project review.

The network representation links the current grid cell to neighbouring information, radar evidence, AI knowledge, cell classification and next-move selection. This illustrates the information flow used by the reasoning process.

10. Running Model Description and Key Findings

Metric	Reported result / range	Interpretation
Number of runs	128	Repeated runs form the analysis dataset.
Cells inspected	17–68; median ≈ 45	Substantial variation in search effort.
Safe moves	4–55; median ≈ 35	Availability of logical safe moves varies.
Fallback moves	0–23; median ≈ 11	Fallback exploration is used when no guaranteed safe move is available.
Thieves identified	0–5; median ≈ 2	Number identified differs between runs.
Total moves	17–68; median ≈ 45	Movement count varies similarly to inspected cells.
Execution time	Approximately 0.5–1.1 seconds	Reported variation is comparatively low.
Control chart	Mean 43.08; UCL 83.49; LCL 2.66	All plotted values reportedly fall within limits.

- Cells inspected and total moves have a reported perfect positive correlation (1.00).
- Execution time is strongly associated with cells inspected and total moves in the reported Pearson and Spearman analyses.
- The control-chart analysis reports that all plotted Total Moves values remain within calculated control limits.
- Fallback moves and thieves identified show little association in the reported Pearson, Spearman and Kendall analyses.
- The decision tree provides interpretable conditions for classifying run outcomes as Failure or No Failure.

Correlation describes association and does not establish causation. Reported results should be verified against the current CSV before submission if the dataset has changed.

11. Project Risks and Mitigations

Risk	Mitigation described in project material
Incorrect or weak search decisions	Use propositional logic and constraint propagation to infer candidate safe cells.
No guaranteed safe move	Use a fallback/probabilistic sweep so the search can continue.
Neighbour-information problem	Calculate adjacent-thief density and use neighbouring-cell information.
Insufficient proximity information	Use Manhattan-distance-based Hot/Cold radar feedback.
Limited search environment	Use the expanded 16 × 16 grid for testing.
Poor visual feedback	Display scanned cells, safe/cleared cells, captured fugitives and radar feedback.
Insufficient test runs	Run more scenarios and inspect variation across repeated runs.

12. Project Execution Screenshots

The following screenshots show the user interface and example execution states supplied with the project.

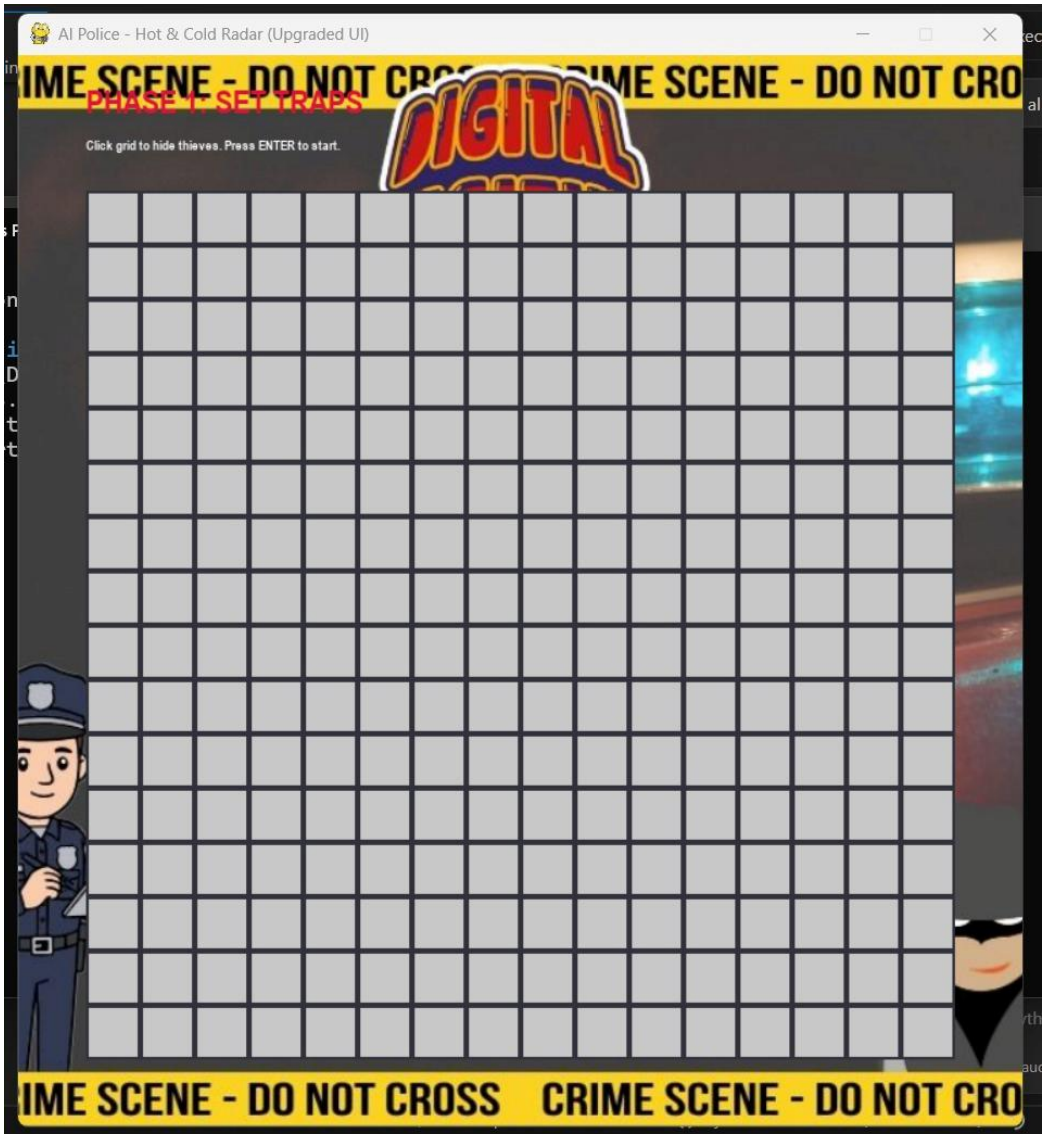


Figure 11. Initial grid interface before fugitive placement.

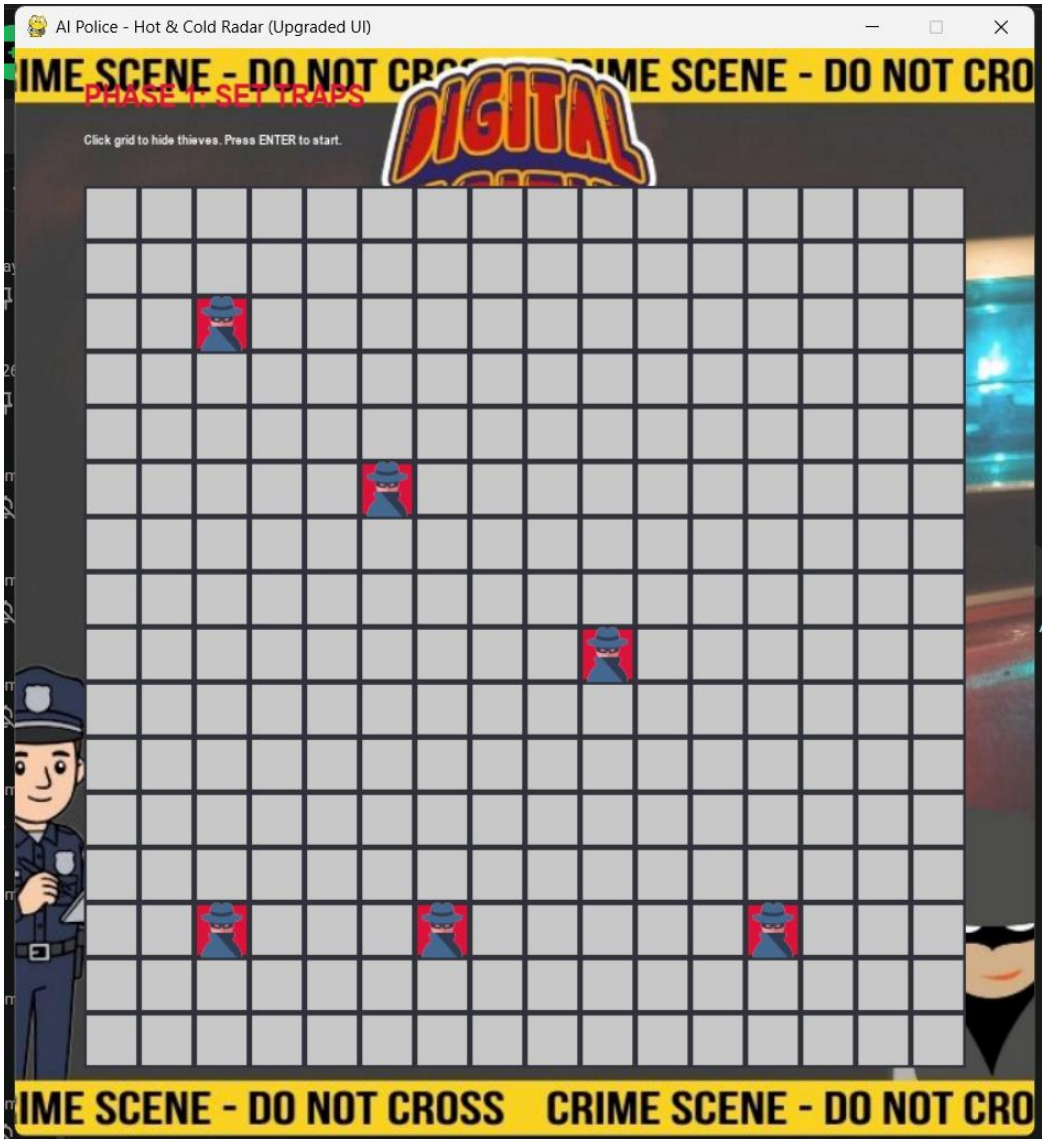


Figure 12. Phase 1: sample hidden-fugitive placement in the grid.



Figure 13. Phase 2: radar-tracking view with cell states and feedback.



Figure 14. Alternate radar-tracking state illustrating search feedback.

13. Conclusion

Digital Fugitive demonstrates an end-to-end grid-based search process in which the environment, reasoning components and visual feedback work together. The Pygame interface makes the search state observable, while the knowledge base, propositional logic and constraint propagation provide a structured way to reason about visited, safe and suspected cells. Manhattan-distance-based Hot/Cold radar adds proximity information, and fallback exploration allows the simulation to continue when the available evidence does not establish a guaranteed safe move.

The software-metrics analysis adds a quantitative view of the simulation. Across the supplied 128-run dataset, cells inspected and total moves have a reported perfect positive correlation of 1.00, while execution time is strongly associated with search effort in the reported correlation matrices. The control chart reports a mean of 43.08 total moves, with UCL = 83.49 and LCL = 2.66; the supplied analysis states that the plotted values remain within these limits. The decision tree provides

interpretable conditions for classifying Failure versus No Failure. These findings help describe the dataset, but they do not establish that one factor causes another, nor do they prove real-world surveillance effectiveness.

In conclusion, the project is valuable as a demonstrator of AI search reasoning and a practical application of software-metrics methods. Further work should strengthen reproducibility, evaluate search strategies under comparable conditions, validate the classifier on held-out runs, and extend the metrics collected. The reported values should be checked against the current CSV if the dataset changes.

14. Future Enhancements

The following enhancements are proposed extensions; they are not claimed as completed features in the current implementation.

Proposed enhancement	Why it is useful	Expected benefit
Automated dataset export	Manually collecting run results can introduce omissions or inconsistent formatting.	Save each run and batch summary directly to CSV with consistent column names.
Controlled strategy benchmarking	Random and knowledge-based search should be compared under equivalent conditions.	Use the same grid layouts, target counts and random seeds to compare search effort and outcomes fairly.
More runs and confidence intervals	A larger sample can make the estimates of typical behaviour more stable.	Report means/medians, spread and confidence intervals across more runs and grid sizes.
Classifier validation	A decision tree can overfit if it is evaluated only on the data used to train it.	Use a stratified train/test split or cross-validation; report confusion matrix, precision, recall and F1-score.
Reproducible run configuration	The ability to repeat a scenario helps verify a reported result.	Record random seed, grid size, target placement, search mode and configuration for each run.
Broader runtime instrumentation	Execution time alone does not describe every aspect of performance.	Where supported, record CPU use, memory, frame rate and visualization latency.
Improved monitoring dashboard	A clear summary can help users interpret a batch without opening separate files.	Add a legend, progress indicator, pause/reset controls and charts for key metrics.

15. References

- Digital Fugitive project report, “Software Metrics,” REVIEW 1 PDF.
- Digital Fugitive project review, “Software Metrics — Fall Semester 2026–27,” supplied Word document.
- Digital Fugitive project presentation, supplied PowerPoint.
- Dataset: Digital Fugitive 128-Run Dataset with Execution Time (CSV). https://drive.google.com/file/d/1uKCd2XEqziQDHPWwrbL6zTwGfSY_vXxP/view?usp=drivesdk
- Source code repository: Digital-Fugitive. https://github.com/sooriya-moorthy-0107/Digital-Fugitive.git